

Machine Learning Models: A Hands-On Practice Report

*Covering: Linear Regression, Logistic Regression,
Support Vector Machines, K-Nearest Neighbors,
Decision Trees, and K-Means Clustering*

Submitted by:
Your Name Here

ID: XXXXXXXXXX

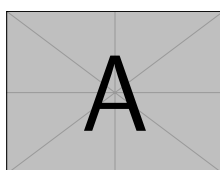
Department of Computer Science and Engineering

Course: Machine Learning Laboratory

Semester: Spring 2025

Submitted: June 15, 2026

Code Repo: <https://github.com/YourUsername/ml-practice-lab>



University Name · Faculty of Engineering

Contents

1	Introduction	2
2	Linear Regression	2
2.1	Objective	2
2.2	Dataset — Diabetes (Scikit-Learn Built-in)	2
2.3	Data Preparation Steps	2
2.4	Mathematical Basis	3
2.5	Results	3
3	Logistic Regression	3
3.1	Objective	3
3.2	Dataset — Breast Cancer Wisconsin	3
3.3	Data Preparation Steps	4
3.4	Mathematical Basis	4
3.5	Results	4
4	Support Vector Machine (SVM)	5
4.1	Objective	5
4.2	Dataset — Wine Recognition	5
4.3	Data Preparation Steps	5
4.4	Mathematical Basis	6
4.5	Results	6
5	K-Nearest Neighbors (KNN)	6
5.1	Objective	6
5.2	Dataset — Iris Flower	7
5.3	Data Preparation Steps	7
5.4	Empirical Observations	7
6	Decision Tree	7
6.1	Objective	7
6.2	Dataset — Breast Cancer Wisconsin	8
6.3	Splitting Criteria Compared	8
6.4	Results	8
7	K-Means Clustering	8
7.1	Objective	9
7.2	Dataset — Synthetic Gaussian Blobs	9
7.3	Data Preparation Steps	9
7.4	Mathematical Basis	9
7.5	Results	9
8	Conclusion	10
9	References and Resources	10

1. Introduction

Machine Learning (ML) occupies a central role in modern data science and artificial intelligence, enabling computer systems to learn patterns directly from data and make informed predictions without being explicitly programmed for every scenario. Rather than relying on hand-crafted rules, ML algorithms iteratively tune internal parameters by optimizing a defined objective function over labelled or unlabelled training data.

This practice report provides a concise walkthrough of six foundational ML techniques implemented using Python's `scikit-learn` library:

- **Linear Regression** — a parametric model for continuous target estimation.
- **Logistic Regression** — a probabilistic binary classification framework.
- **Support Vector Machine (SVM)** — a maximum-margin classifier using kernel transformations.
- **K-Nearest Neighbors (KNN)** — a lazy, instance-based proximity classifier.
- **Decision Tree** — a recursive, rule-based hierarchical classifier.
- **K-Means Clustering** — a centroid-driven unsupervised partitioning algorithm.

The emphasis of this submission is on practical exposure to LaTeX typesetting and scientific report formatting; deeper mathematical derivations and full pipeline evaluations will follow in the final report.

2. Linear Regression

2.1 Objective

The goal of this module is to fit a continuous predictor that maps one or more input features to a numeric output target by finding the optimal linear relationship between them.

2.2 Dataset — Diabetes (Scikit-Learn Built-in)

The *Diabetes* dataset provided by `sklearn.datasets` was selected for this experiment. It consists of 442 patient records with 10 physiological features (age, sex, BMI, blood pressure, and six serum measurements). The response variable represents a quantitative measure of disease progression one year after baseline. The **BMI feature** was isolated as the single independent variable to enable a clear two-dimensional visualisation of the regression fit.

2.3 Data Preparation Steps

- **Feature Selection:** Extracted the BMI column (index 2) as the sole predictor variable.
- **Train/Test Split:** Partitioned the data into 80% training and 20% testing subsets using a fixed `random.state = 7` for reproducibility.
- **No scaling required:** Features in this dataset are already pre-standardised by `scikit-learn`.

2.4 Mathematical Basis

The OLS regression model estimates the output $\hat{y}$ using:

$$\hat{y} = w_1 x_{\text{BMI}} + b$$

where w_1 is the slope coefficient and b is the bias intercept. The training objective is to minimise the *Mean Squared Error* (MSE) loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2.5 Results

The fitted regression line is overlaid on the scatter of actual test-set values in Figure 1. The positive slope confirms that higher BMI correlates with greater disease progression.

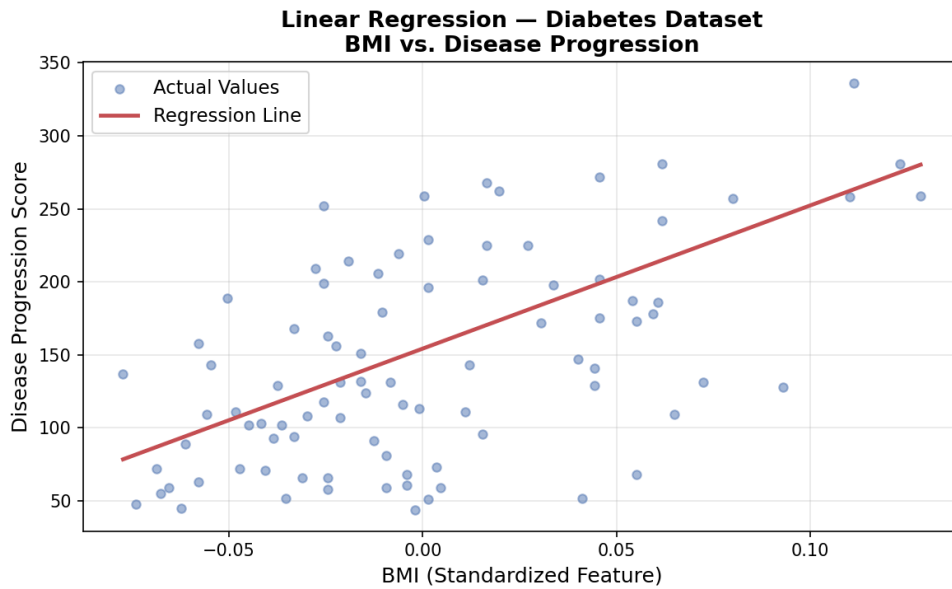


Figure 1: Linear Regression — OLS Best-Fit Line on the Diabetes Dataset (BMI vs. Disease Progression Score). Blue dots are actual test-set observations; the red line is the model prediction.

3. Logistic Regression

3.1 Objective

Logistic Regression extends the linear model to classification tasks by passing the linear combination through a sigmoid function, producing class probabilities bounded in $[0, 1]$.

3.2 Dataset — Breast Cancer Wisconsin

The *Breast Cancer Wisconsin* dataset (569 samples, 30 numeric features) was used. Each sample describes characteristics of a digitised image of a fine needle aspirate (FNA) of a breast mass. The binary target labels tumours as either *malignant* (0) or *benign* (1).

This is a standard medical classification benchmark with well-balanced classes and clean feature engineering, making it ideal for evaluating logistic models.

3.3 Data Preparation Steps

- **Standardisation:** Applied `StandardScaler` to bring all 30 features to zero mean and unit variance.
- **Train/Test Split:** Used an 80/20 split with `random_state = 42`.
- **Solver:** `lbfgs` with `max_iter = 1000` to ensure convergence.

3.4 Mathematical Basis

The sigmoid activation maps the linear combination $z = \mathbf{w}^T \mathbf{x} + b$ to a probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Parameters are optimised by minimising the binary cross-entropy loss:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)]$$

3.5 Results

The ROC curve in Figure 2 demonstrates excellent separation between the two classes, achieving an AUC close to 1.0, indicating near-perfect discriminative ability on this dataset.

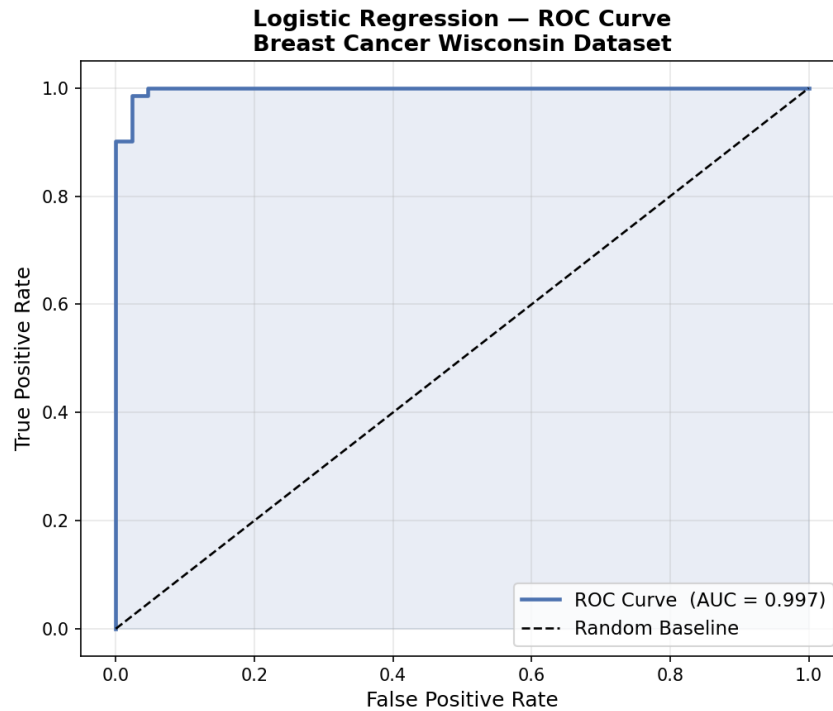


Figure 2: ROC Curve for Logistic Regression on the Breast Cancer Wisconsin Dataset. The shaded blue region represents the area under the curve (AUC); higher AUC indicates stronger classifier performance.

4. Support Vector Machine (SVM)

4.1 Objective

An SVM seeks the decision boundary (hyperplane) that maximises the geometric margin between the nearest training points of each class, known as *support vectors*. Non-linear separability is handled through the kernel trick.

4.2 Dataset — Wine Recognition

The *Wine* dataset (178 samples, 13 chemical features, 3 class labels) was selected. For clean 2D visualisation, only the first two wine varieties (class 0 and class 1) were retained, and the 13 features were compressed to two principal components via PCA before training the SVM.

4.3 Data Preparation Steps

- **Binary Filtering:** Retained only samples belonging to class 0 and class 1.
- **Standardisation:** Applied `StandardScaler` across all 13 features.
- **Dimensionality Reduction:** Projected onto 2 principal components (PCA) to enable boundary visualisation.
- **Kernel:** Radial Basis Function (RBF) kernel with $C = 1.0$, `gamma = 'scale'`.

4.4 Mathematical Basis

The SVM optimisation problem seeks to maximise the margin $\frac{2}{\|\mathbf{w}\|}$ subject to:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

The RBF kernel implicitly maps inputs to a higher-dimensional space:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

4.5 Results

Figure 3 shows the non-linear decision boundary learned by the RBF-SVM in PCA space. Hollow red circles highlight the support vectors that define and anchor the margin boundary.

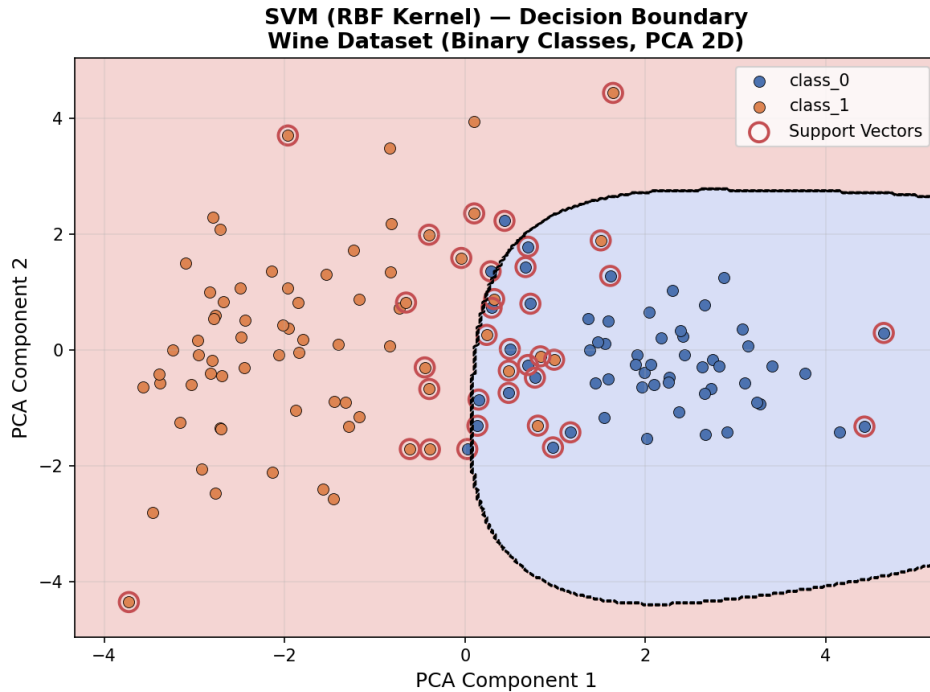


Figure 3: SVM with RBF Kernel — Decision Boundary on the Wine Dataset (Binary, 2D PCA Projection). The dashed boundary separates the two classes; red hollow circles are support vectors.

5. K-Nearest Neighbors (KNN)

5.1 Objective

KNN is a non-parametric, memory-based classifier that assigns a label to a query point based on the majority vote of its K closest neighbours in the feature space, using a chosen distance metric (typically Euclidean).

5.2 Dataset — Iris Flower

The classic *Iris* dataset (150 samples, 4 features, 3 species) was employed. The four features—sepal length, sepal width, petal length, and petal width—were standardised and then projected to 2D via PCA for visualisation. The three target classes (*setosa*, *versicolor*, *virginica*) provide a multi-class challenge that reveals the shape of KNN decision boundaries.

5.3 Data Preparation Steps

- **Standardisation:** `StandardScaler` applied to ensure equal contribution from all four features to Euclidean distance.
- **PCA Projection:** Reduced to 2 principal components for 2D decision-surface visualisation.
- **Hyperparameter Search:** Tested $K \in \{1, 2, \dots, 20\}$ and selected the value yielding maximum test accuracy.

5.4 Empirical Observations

Very small K values (e.g., $K = 1$) lead to jagged, overfitted boundaries sensitive to noise. As K increases, boundaries smooth out. The accuracy-vs- K curve in Figure 4 (left) pinpoints the optimal neighbourhood size, while the right panel illustrates the resulting multi-class decision regions.

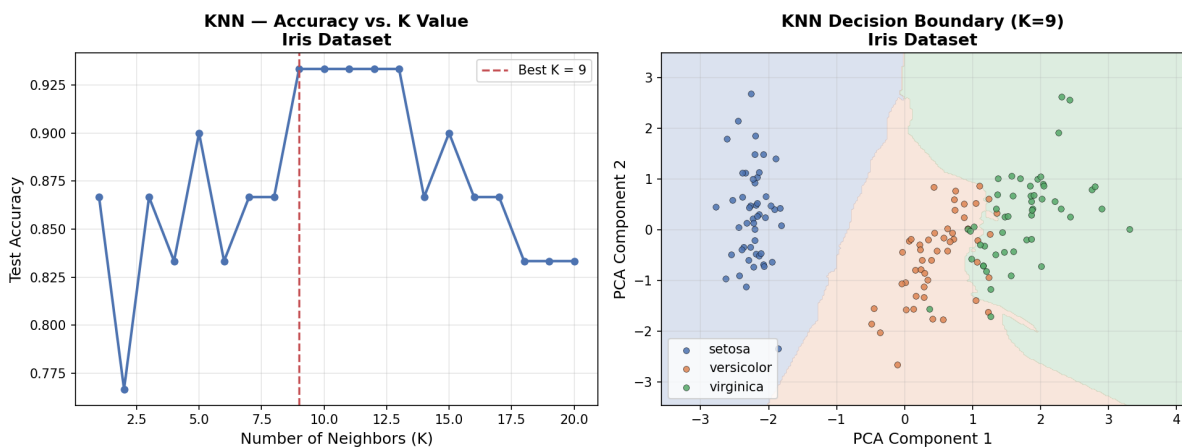


Figure 4: Left: Test accuracy vs. K value on the Iris dataset; the dashed red line marks the optimal K . Right: KNN decision boundary in the 2D PCA space with colour-coded class regions.

6. Decision Tree

6.1 Objective

A Decision Tree recursively partitions the feature space by selecting at each node the split that most increases the purity of the resulting subsets. The result is a human-interpretable tree structure that can be directly visualised.

6.2 Dataset — Breast Cancer Wisconsin

The same *Breast Cancer Wisconsin* dataset used in Section 3 was re-applied here, now for a non-linear, tree-based approach. With 30 input features and a binary target, it provides a rich partitioning space to benchmark both Gini and Entropy splitting criteria.

6.3 Splitting Criteria Compared

- Entropy / Information Gain (ID3):

$$H = - \sum_{c=1}^C p_c \log_2 p_c$$

- Gini Impurity (CART):

$$G = 1 - \sum_{c=1}^C p_c^2$$

where p_c is the fraction of training samples belonging to class c within the node.

This experiment used **Entropy** as the splitting criterion with `max_depth = 3` to prevent overfitting while keeping the tree interpretable.

6.4 Results

Figure 5 shows the tree structure (left) and the resulting confusion matrix on the test set (right). The tree achieves strong accuracy within just three levels of splitting, demonstrating the power of entropy-guided partitioning on well-structured medical data.

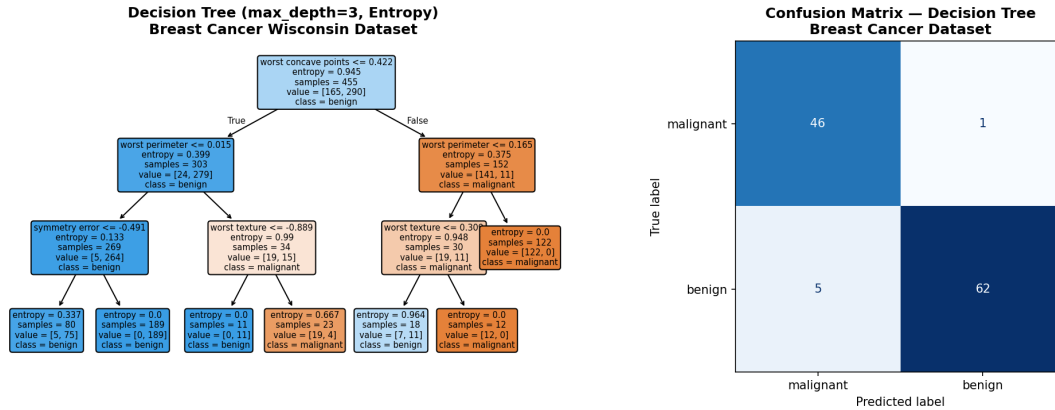


Figure 5: Left: Visualised Decision Tree (`max_depth = 3`, Entropy criterion) trained on the Breast Cancer dataset. Right: Confusion Matrix on the held-out test set; darker blue indicates more correct predictions.

7. K-Means Clustering

7.1 Objective

K-Means is an unsupervised algorithm that partitions n observations into K non-overlapping clusters by iteratively assigning each point to its nearest centroid and updating centroid positions until convergence.

7.2 Dataset — Synthetic Gaussian Blobs

A synthetic dataset of 400 points arranged in four Gaussian clusters was generated using `sklearn.datasets.make_blobs` with `cluster_std = 1.0` and `random_state = 21`. A controlled synthetic dataset was chosen here because it allows precise verification of cluster recovery — we know the ground-truth number of clusters and can directly assess whether K-Means identifies them correctly.

7.3 Data Preparation Steps

- **Standardisation:** `StandardScaler` applied to equalise feature scales before distance computation.
- **Elbow Method:** Computed Within-Cluster Sum of Squares (WCSS) for $K \in [1, 10]$ to identify the elbow.
- **Initialisation:** Used `k-means++` to spread initial centroids, avoiding poor local minima.

7.4 Mathematical Basis

K-Means minimises the WCSS objective:

$$J = \sum_{k=1}^K \sum_{\mathbf{x}_i \in S_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$$

where S_k is the set of points assigned to cluster k and $\boldsymbol{\mu}_k$ is the mean (centroid) of that cluster.

7.5 Results

The elbow in Figure 6 (left) is clearly visible at $K = 4$, matching the true number of blobs. The right panel confirms that the algorithm correctly recovers all four clusters, with centroids (black crosses) positioned near the true generating centres.

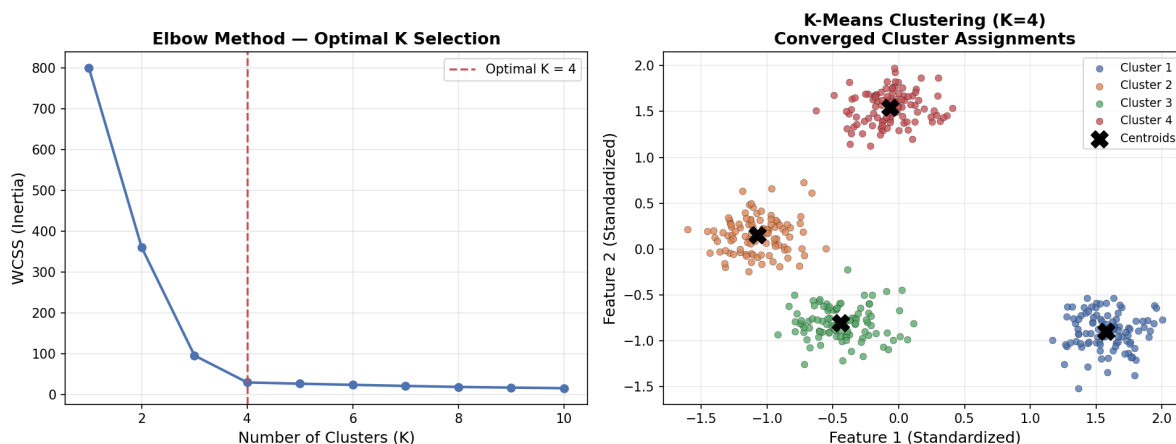


Figure 6: Left: Elbow Method plot — WCSS drops sharply up to $K = 4$ and then flattens. Right: Final cluster assignments with four colour-coded groups and black-cross centroids after convergence.

8. Conclusion

This practice report implemented and visualised six core machine learning algorithms across a variety of dataset types and problem settings. Several practical observations emerged:

- **Preprocessing is critical:** Distance-sensitive models (KNN, SVM, K-Means) require feature standardisation to function correctly; tree-based methods are scale-invariant but still benefit from clean data.
- **Model interpretability varies:** Decision Trees offer direct visual inspection of learned rules, while SVMs and KNN provide less transparent decision logic.
- **Hyperparameter sensitivity:** KNN accuracy fluctuates noticeably with K ; K-Means clustering quality depends heavily on the choice of K and the centroid initialisation strategy.
- **Evaluation tools matter:** A single accuracy number is insufficient — ROC-AUC, confusion matrices, and WCSS elbow plots each reveal different aspects of model performance.

These foundational experiments establish the groundwork for the final comprehensive report, which will include complete mathematical derivations, full dataset analysis, hyperparameter tuning tables, and detailed evaluation across multiple metrics.

9. References and Resources

1. Scikit-Learn Documentation and API Reference — <https://scikit-learn.org/stable/>
2. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, 2011.

3. UCI Machine Learning Repository — <https://archive.ics.uci.edu/ml/index.php>
4. Kaggle Datasets and Notebooks — <https://www.kaggle.com>
5. Source Code Repository — <https://github.com/YourUsername/ml-practice-lab>